

How to Run HotGauge

HotGauge is a toolchain comprised of three parts. The first is SniperSim, which is the performance model, the second is McPAT, the power model, and the third is 3d-ice, the thermal model. HotGauge simulations progress through these models sequentially in that order. Unfortunately, the outputs of McPAT do not perfectly match the inputs to 3d-ice, so there is an intermediate processing step built into the toolchain. With four distinct steps (five if one counts analysis), each with unique inputs and outputs, it is easy to get confused, so it is worth explaining how the toolchain is linked together.

1. SniperSim Inputs/Outputs

The first step in the toolchain is SniperSim. SniperSim takes in many input arguments, outlined in the SniperSim [manual](#), but a typical run looks like this:

```
./run-sniper \  
-c <path to configuration file>  
-n <number of cores to simulate>  
-d <path to output directory>  
-s energystats:40000  
-s stop-by-icount:20000000  
--sde-arch=tgl  
-- <path to workload executable>
```

The arguments mean:

- **-c:** This is the path to your configuration file, which describes the core your workload will run on.
- **-n:** This is an integer representing how many cores you are simulating.
- **-d:** This is the path to your output directory.
- **-s energystats:40000:** This enables the energystats hook and passes it an integer as its interval/argument. This samples/logs energy/power every time interval you define. One interval used in previous studies is 40000.
- **-s stop-by-icount:20000000:** This tells SniperSim how many instructions to simulate. 20,000,000 instructions is usually the minimum required to see hotspots.
- **--sde-arch=tgl:** This tells SniperSim to emulate a Tiger Lake ISA/micro-ops profile.
- **--:** This is followed by the path to the program you would like SniperSim to run on the simulated cores.

SniperSim output is a collection of timestamped energy statistics files that use the name **energystats-temp** along with some additional information about the simulation.

These **energystats-temp** files are fed into McPAT, which turns these timestamped performance metrics files into timestamped power metrics files. The scripts that do this live in `~/HotGauge/scripts`.

After McPAT, the Sniper output directory contains both the original energy-statistics files and the generated power-related outputs.

2. 3d-ice Inputs/Outputs

The next step is 3d-ice, the thermal model. More information about 3d-ice can be found in the 3d-ice [user guide](#). In HotGauge, 3d-ice takes in these inputs:

1. A warmup **.tstack** file. This warmup file is filled with the temperature values that represent the temperature in different parts of the chip at the beginning of the simulation.
2. A stack (**.stk**) description file that describes the thermal problem that 3d-ice will solve. It contains information such as:
 - a) Information about the structure of the die
 - b) Information about its material properties
 - c) Information about heat sinks in the system
 - d) Discretization parameters
 - i) How the die is broken into “thermal cubes” assumed to have uniform temperature
 - e) Analysis parameters
 - i) How simulation is run over time
 - f) Desired outputs
3. A floorplan (**.flp**) file that describes the architecture of the chip and cooling system as seen by the thermal model. It knows where each major logic block is and how big it is.
4. A power trace file. This file is created from the **block_powers** files that are the output of McPAT. The values inside it represent how much power different parts of the chip are using over time. These values are streamed into 3d-ice, which uses them to calculate temperature values.
5. A metadata file. This file contains information about the HotGauge run, including the name of the run, the frequency at which the Sniper simulation ran, and the instruction count.

3. Example of a Full HotGauge Run

This section walks through one example of a full HotGauge run.

3.1 SniperSim

Navigate to `~/HotGauge/examples` and open the **skylake_14nm.cfg** file. The top of the file should be edited so that the frequency is set to 4.7.

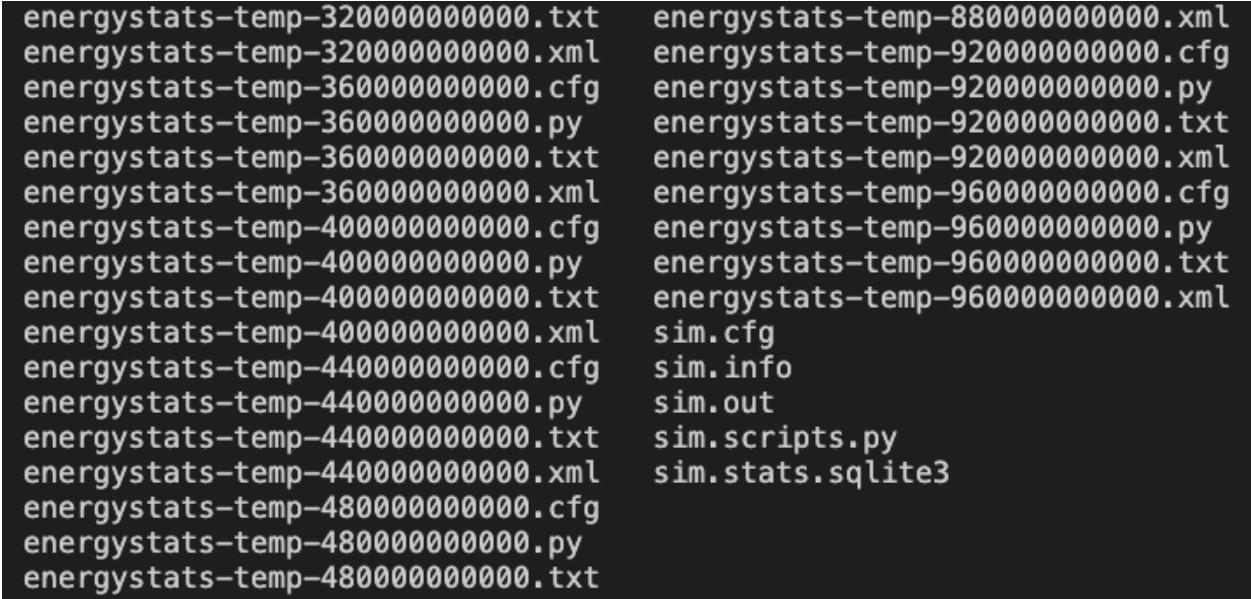
Next, compile the modified Linpack benchmark we will use for this run:

```
gcc -O -o mod_linpack modified_linpack.c -lm
```

Navigate back to ~/HotGauge/snipersim and start sniper:

```
./run-sniper -c ~/HotGauge/examples/skylake_14nm.cfg -n 7 -d \  
~/HotGauge/snipersim/output/mod_linpack/14nm/4.7Ghz -s energystats:40000 -s \  
stop-by-icount:20000000 --sde-arch=tgl -- \  
~/HotGauge/examples/mod_linpack
```

When the simulation ends, double check that in ~/HotGauge/snipersim/output/mod_linpack/14nm/4.7Ghz you see a bunch of time-stamped **energystats-temp** files. The output should look like this, though with more files:



```
energystats-temp-320000000000.txt  energystats-temp-880000000000.xml  
energystats-temp-320000000000.xml  energystats-temp-920000000000.cfg  
energystats-temp-360000000000.cfg  energystats-temp-920000000000.py  
energystats-temp-360000000000.py  energystats-temp-920000000000.txt  
energystats-temp-360000000000.txt  energystats-temp-920000000000.xml  
energystats-temp-360000000000.xml  energystats-temp-960000000000.cfg  
energystats-temp-400000000000.cfg  energystats-temp-960000000000.py  
energystats-temp-400000000000.py  energystats-temp-960000000000.txt  
energystats-temp-400000000000.txt  energystats-temp-960000000000.xml  
energystats-temp-400000000000.xml  sim.cfg  
energystats-temp-440000000000.cfg  sim.info  
energystats-temp-440000000000.py  sim.out  
energystats-temp-440000000000.txt  sim.scripts.py  
energystats-temp-440000000000.xml  sim.stats.sqlite3  
energystats-temp-480000000000.cfg  
energystats-temp-480000000000.py  
energystats-temp-480000000000.txt
```

Figure 1: Snipersim Output Example

3.2 McPAT

Navigate to ~/HotGauge/scripts.

Then run:

```
./14_perf_sims_to_7_10_14_power_sims.sh \  
~/HotGauge/snipersim/output/mod_linpack 7
```

Inside of ~/HotGauge/snipersim/output/mod_linpack/7nm/4.7GHz you should see a bunch of time-stamped **energystats-temp**, **mcpat_output**, and **block_powers** files. It should look like this:

block_powers_2120000000000.json	energystats-temp-2160000000000.xml	mcpat_output_160000000000.json	mcpat_output_520000000000.json
block_powers_2160000000000.json	energystats-temp-2200000000000.xml	mcpat_output_160000000000.txt	mcpat_output_520000000000.txt
block_powers_2200000000000.json	energystats-temp-2240000000000.xml	mcpat_output_160000000000.json	mcpat_output_560000000000.json
block_powers_2240000000000.json	energystats-temp-2280000000000.xml	mcpat_output_160000000000.txt	mcpat_output_560000000000.txt
block_powers_2280000000000.json	energystats-temp-2320000000000.xml	mcpat_output_1640000000000.json	mcpat_output_600000000000.json
block_powers_2320000000000.json	energystats-temp-2360000000000.xml	mcpat_output_1640000000000.txt	mcpat_output_600000000000.txt
block_powers_2360000000000.json	energystats-temp-2400000000000.xml	mcpat_output_1680000000000.json	mcpat_output_640000000000.json

Figure 2: Example McPAT output

3.3 3d-ice

Navigate to `~/HotGauge/examples` and create a new directory for your run. Copy the contents of the template directory into it.

For Example:

```
mkdir mod_linpack_dir
cp -r template/* mod_linpack_dir/
```

Enter `/HotGauge/examples/mod_linpack_dir`.

The first step is to create the power trace and metadata files. The script that does this is `make_pow_trace.py`. The script calls the power trace file `power_trace.json` and the metadata file `metadata.json`. You can use the `--prefix-for-files` in the command line to add a prefix to those two file names.

Run:

```
python make_pow_trace.py --sniper-output-dir \
~/HotGauge/snipersim/output/mod_linpack/7nm/4.7Ghz --prefix-for-files prefix \
--instruction-count 20000000 --interval-ns 40000 --suite linpack
```

There should now be two new directories, one called `Metadata` and one called `Traces`. The `Metadata` directory will have your metadata file, which will be called `prefix_metadata.json`, and the `Traces` directory will have your power trace file, which will be called `prefix_pow_trace.json`.

The next step is to generate a warmup. The script that does this is called `generate_warmup.py`. If you would like to change the heatsink model, you will need to edit the script.

The script can be run like this:

```
python generate_warmup.py --warmup-dir . --flp-template \
skylake7nm_7core_3_3D-ICE_template.flp
```

This script can take days to run, so for convenience, HotGauge ships with an `example_warmup` directory that can be used to finish this run.

The last step in 3d-ice is to run `sim_from_warmup.py`. Before we run this however, you will want to make sure:

- the `sys.path.insert` lines are correct,
- and the heatsink information matches that of your warmup.

The other important thing to note before running the simulation script is what you would like your core mapping to be. Core mapping is sent in from the command line, and different commands correspond to different mappings:

- Passing 0,1,3 causes cores 0, 1, and 3 to be present in the simulation. Each of these cores receives its power statistics from the corresponding core in Sniper.
- Passing 0->2,6 causes cores 0, 2, and 6 to be present in the simulation, but each of those cores receives its power statistics from core 0 of the Sniper simulation.

Once that has been confirmed, you can run:

```
python sim_from_warmup.py --tstack-path \
~/HotGauge/examples/mod_linpack_dir/example_warmup/skylake7nm_7core_3_3D-ICE_template.flp
/final.tstack \
--flp-template skylake7nm_7core_3_3D-ICE_template.flp \
--trace-file ~/HotGauge/examples/mod_linpack_dir/Traces/prefix_pow_trace.json \
--metadata-file ~/HotGauge/examples/mod_linpack_dir/Metadata/prefix_metadata.json \
--core-mapping '0->2,6'
```

If you would like to run multiple simulations, you can pass `-workload-manifest {path to workload.json}` instead of a single metadata file and trace file.

That `workload.json` file can be created by running:

```
python make_workload_json.py
```

The output `workload.json` file should be structured like this:

```
[
  { "trace": "../Traces/A_pow_trace.json",
    "meta": "../Metadata/A_metadata.json" },
  { "trace": "../Traces/B_pow_trace.json",
    "meta": "../Metadata/B_metadata.json" }
]
```

4. Analysis

Inside your `/HotGauge/examples/mod_linpack_dir` directory, there is a subdirectory called `analysis_scripts`. Inside are three scripts that can each be run like this:

```
python {analysis_script}.py \
--metadata-file-name <name of the metadata file of the 3d-ice run to analyze> \
--core-name <name of the core you are analyzing, usually core_0>
```

The order in which you run the scripts is important.

The `compute_local_maxima_stats.py` script sorts through your 3d-ice output to find hotspot candidates, and it outputs a `die_grid.temps.1dmaxima` and a `die_grid.temps.2dmaxima`. These are taken as input for the other two scripts, `plot_vs_time.py` and `diagram_visualize.py`.

- `plot_vs_time.py` creates two plots:
 - one showing temperature statistics over time,

- and one showing a temperature distribution histogram over time.
- `diagram_visualize.py` outputs a series of images that are linked together to form a video showing hotspot formation over time.

It is important to note that `diagram_visualize.py` takes in additional inputs called `grid-args`. These arguments control the size of the heatmap, legend, and font. Getting a clean image may require some experimentation. For this run, this command will produce a good image:

```
python diagram_visualize.py --metadata-file-name prefix_metadata.json -- \
core-name cores_2_6 --grid-args "--heatmap_width 80.0" --grid-args "-- \
scale_width 8.0" --grid-args "--min_t 40" --grid-args "--max_t 115"
```

Decreasing the `--heatmap_width` lowers the portion of the image produced occupied by the heatmap, which may be necessary to see the temperature labels. It is also a good idea to set `--min_t` and `--max_t` based on the temperatures for which you are looking.

5. End to End

To avoid progressing through this long, annoying process, HotGauge ships with a directory called `end_to_end` that contains a `run_HotGauge.py` script and a configuration file template called `config_template.yaml`. This script runs the entire toolchain but not the analysis scripts. The configuration file contains a `USER EDIT SECTION` where you can fill out the input fields required for the entire HotGauge toolchain. You may also edit the fields below the `USER EDIT SECTION`, but the important things that vary from experiment to experiment are editable from the `USER EDIT SECTION`. You can run the script like this:

```
python run_HotGauge.py --config {your_config.yaml}
```